

K. Ruthwik Bhat

2403a51l08

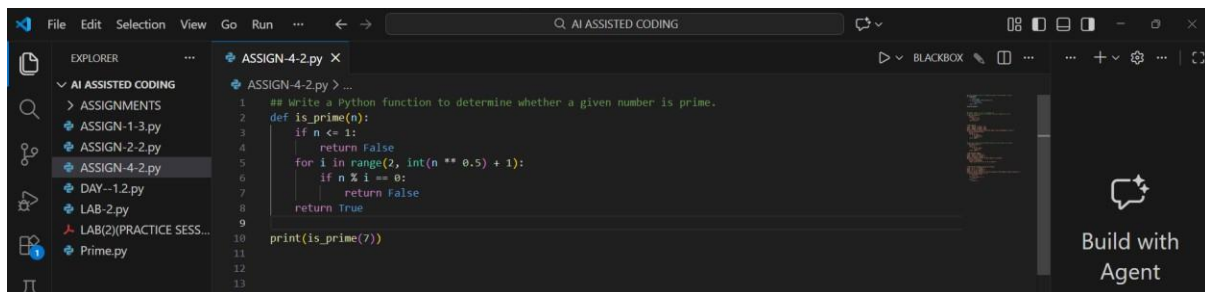
B-51

Lab 4

Advanced Prompt Engineering - Zero-shot, One-shot, and Few-shot Techniques

Task Description-1: Zero-shot Prompting

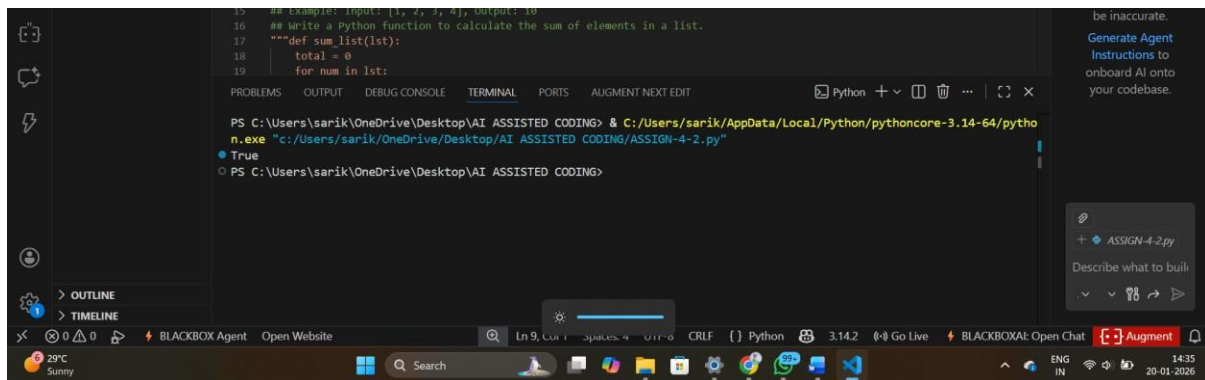
Prompt: Write a Python function to determine whether a given number is prime.



The screenshot shows a code editor with a file explorer on the left. The file explorer lists several files under 'AI ASSISTED CODING', including 'ASSIGN-1-3.py', 'ASSIGN-2-2.py', 'ASSIGN-4-2.py', 'DAY-1.2.py', 'LAB-2.py', 'LAB(2)(PRACTICE SESS...', and 'Prime.py'. The main editor window displays the code for 'ASSIGN-4-2.py'. The code is as follows:

```
1  ## Write a Python function to determine whether a given number is prime.
2  def is_prime(n):
3      if n <= 1:
4          return False
5      for i in range(2, int(n ** 0.5) + 1):
6          if n % i == 0:
7              return False
8      return True
9
10 print(is_prime(7))
11
12
13
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-4-2.py"
True
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

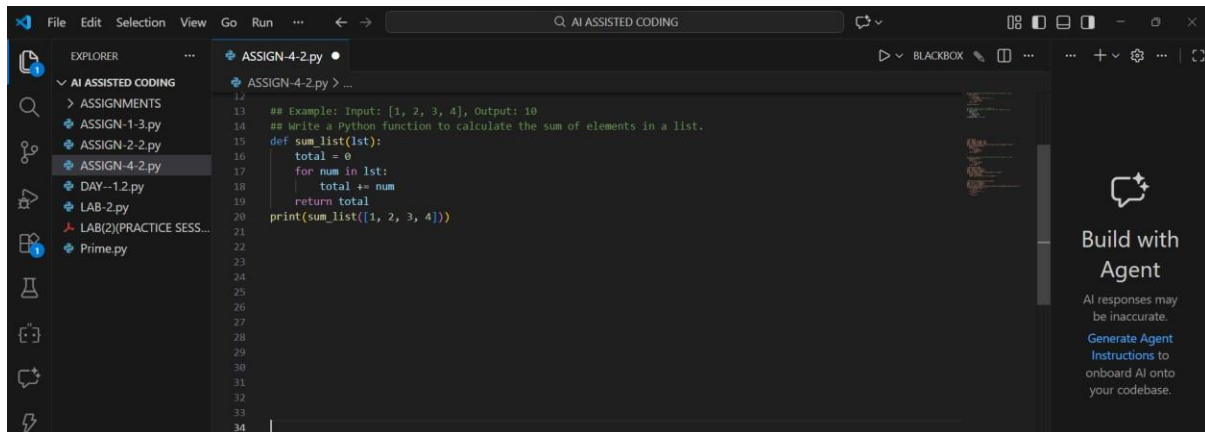
Explanation:

1. Zero-shot prompting provides only instructions, no examples.
2. The AI correctly implemented:
 - Prime definition logic
 - Square-root optimization
3. Demonstrates that simple logical problems work well with zero-shot prompts.

Task Description-2: One-shot Prompting

Prompt: Write a Python function to calculate the sum of elements in a list.

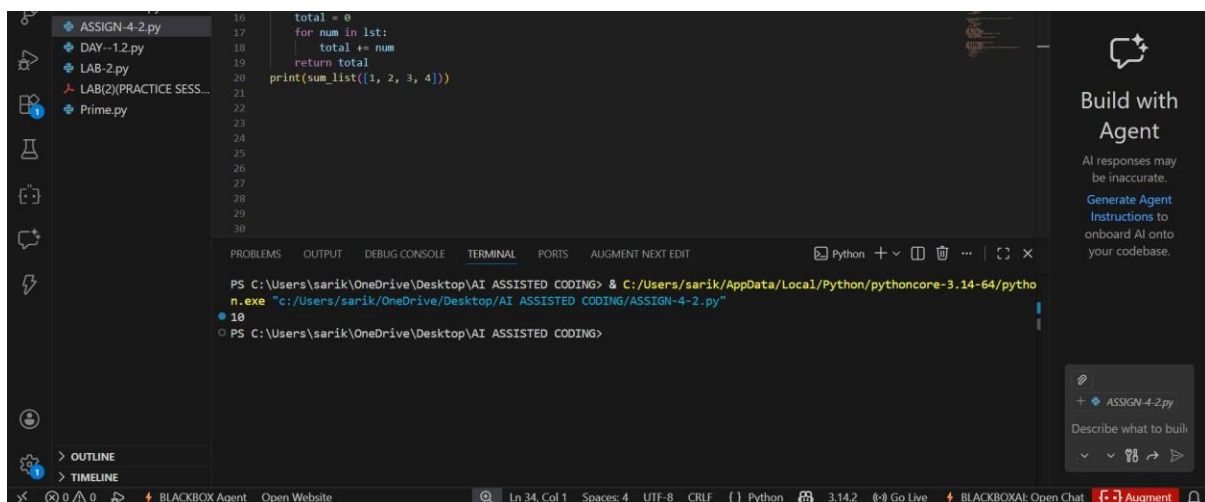
Example: Input: [1, 2, 3, 4], Output: 10



The screenshot shows the Visual Studio Code editor with a file named 'ASSIGN-4-2.py'. The code defines a function 'sum_list' that takes a list 'list' as input, initializes a 'total' variable to 0, and iterates over each element in the list, adding it to the total. Finally, it prints the total. The example input [1, 2, 3, 4] is used to demonstrate the function.

```
12
13 ## Example: Input: [1, 2, 3, 4], Output: 10
14 ## Write a Python function to calculate the sum of elements in a list.
15 def sum_list(list):
16     total = 0
17     for num in list:
18         total += num
19     return total
20 print(sum_list([1, 2, 3, 4]))
21
22
23
24
25
26
27
28
29
30
31
32
33
34
```

OUTPUT:



The screenshot shows the Visual Studio Code editor with the same file 'ASSIGN-4-2.py'. The code is the same as in the previous screenshot. The output of the code is displayed in the terminal window at the bottom, showing the command 'python n.exe' and the output '10'.

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python
n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-4-2.py"
10
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

1. One example clarifies the expected behavior.
2. The AI correctly inferred:

Iteration over list

Accumulation of sum

3. The example helped remove ambiguity.

Task Description-3: Few-shot Prompting

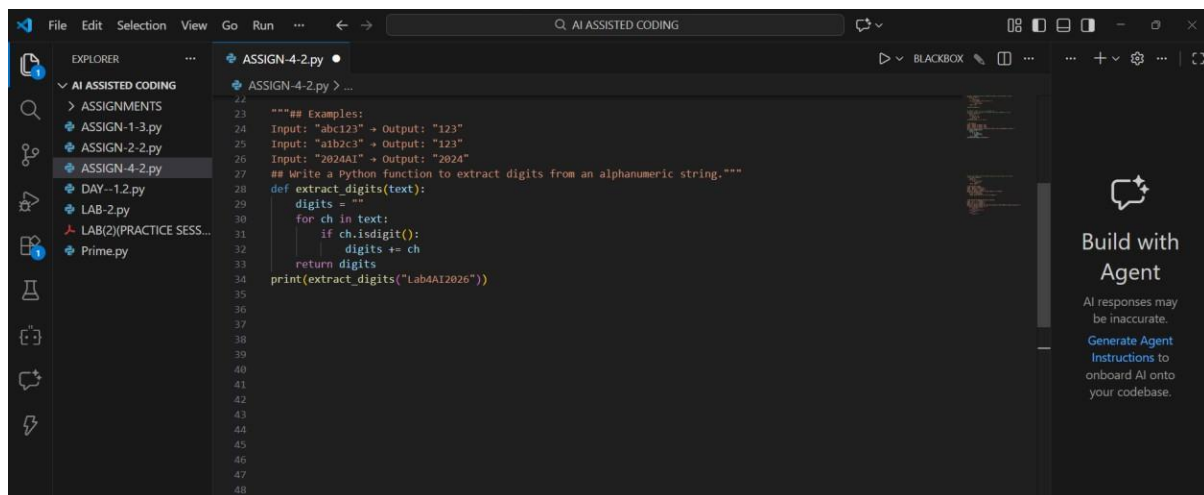
Prompt: Write a Python function to extract digits from an alphanumeric string.

Examples:

Input: "abc123" → Output: "123"

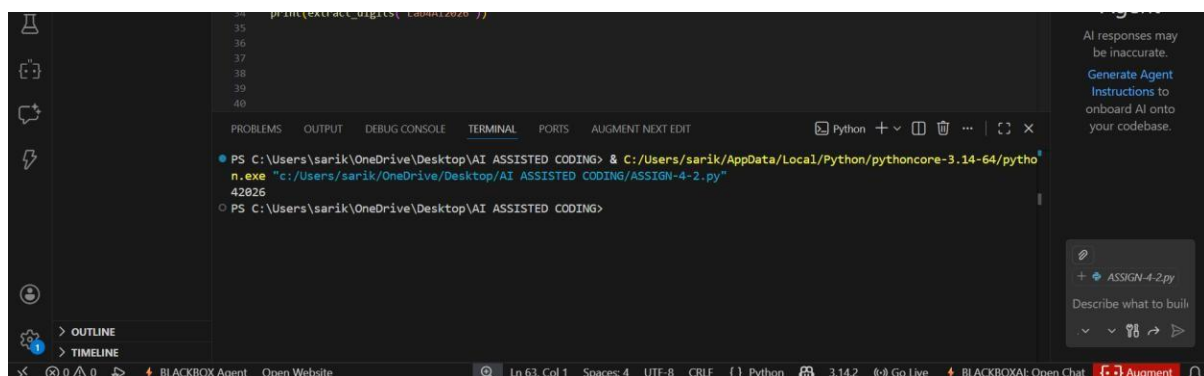
Input: "a1b2c3" → Output: "123"

Input: "2024AI" → Output: "2024"



```
22
23 """## Examples:
24 Input: "abc123" → Output: "123"
25 Input: "a1b2c3" → Output: "123"
26 Input: "2024AI" → Output: "2024"
27 ## Write a Python function to extract digits from an alphanumeric string."""
28 def extract_digits(text):
29     digits = ""
30     for ch in text:
31         if ch.isdigit():
32             digits += ch
33     return digits
34 print(extract_digits("Lab4AI2026"))
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI_ASSISTED_CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI_ASSISTED_CODING/ASSIGN-4-2.py"
42026
PS C:\Users\sarik\OneDrive\Desktop\AI_ASSISTED_CODING>
```

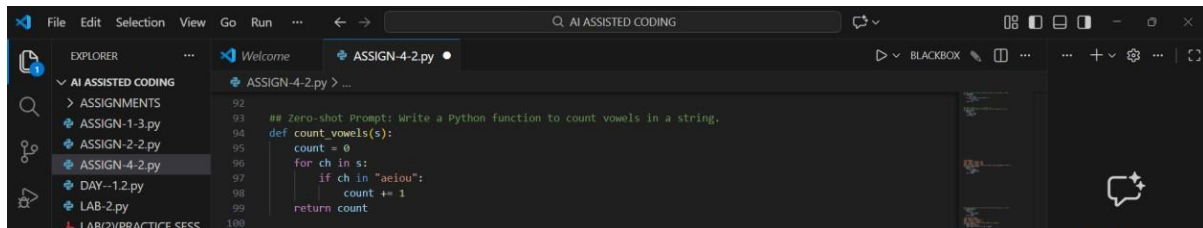
Explanation:

1. Few-shot prompting provides pattern recognition.
2. AI correctly:
 - Identified digit extraction rule
 - Ignored alphabetic characters

3. Output accuracy improved due to multiple examples.

Task Description-4: Comparison Zero-shot vs Few-shot Prompting

Zero-shot Prompt: Write a Python function to count vowels in a string.



The screenshot shows the Visual Studio Code editor with a file named 'ASSIGN-4-2.py'. The code defines a function 'count_vowels(s)' that takes a string 's' as input and returns the count of vowels. The function uses a loop to iterate over each character in the string and increments a counter if the character is a vowel (a, e, i, o, u). The prompt 'Zero-shot Prompt: Write a Python function to count vowels in a string.' is visible above the code.

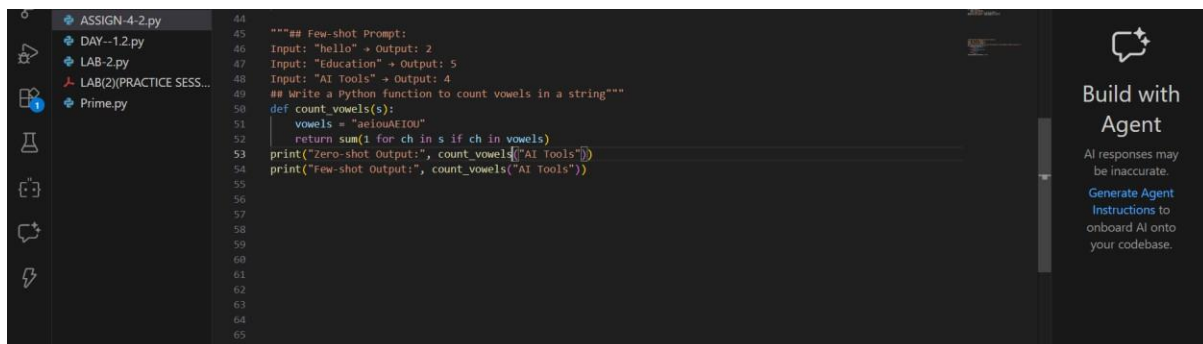
Few-shot Prompt: Write a Python function to count vowels in a string

Examples:

Input: "hello" → Output: 2

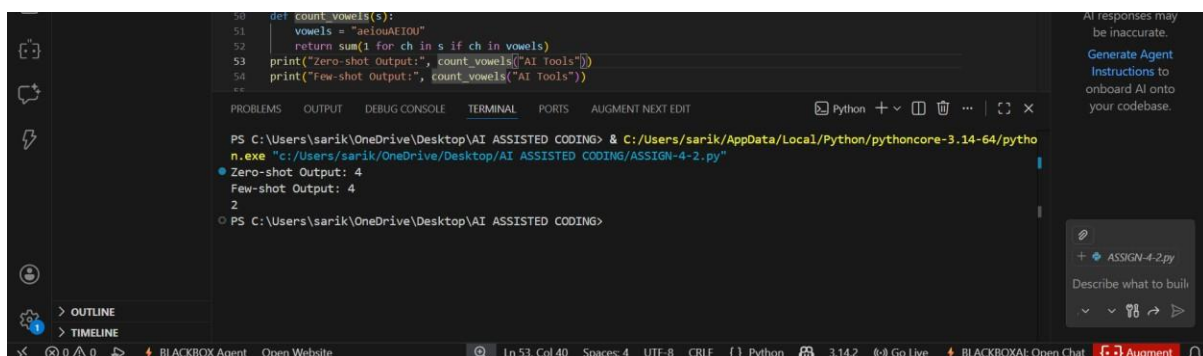
Input: "Education" → Output: 5

Input: "AI Tools" → Output: 4



The screenshot shows the Visual Studio Code editor with a file named 'ASSIGN-4-2.py'. The code defines a function 'count_vowels(s)' that takes a string 's' as input and returns the count of vowels. The function uses a loop to iterate over each character in the string and increments a counter if the character is a vowel (a, e, i, o, u). The prompt 'Few-shot Prompt: Write a Python function to count vowels in a string' is visible above the code, followed by three examples: 'Input: "hello" → Output: 2', 'Input: "Education" → Output: 5', and 'Input: "AI Tools" → Output: 4'. The code then prints the output for 'AI Tools'.

OUTPUT:



The screenshot shows the Visual Studio Code editor with the same file 'ASSIGN-4-2.py'. The code defines a function 'count_vowels(s)' that takes a string 's' as input and returns the count of vowels. The function uses a loop to iterate over each character in the string and increments a counter if the character is a vowel (a, e, i, o, u). The code then prints the output for 'AI Tools'. The output is displayed in the terminal window at the bottom of the editor, showing 'Zero-shot Output: 4' and 'Few-shot Output: 2'.

Comparison Table:

Feature	Zero-shot	Few-shot
Case handling	Only lowercase	Upper C lowercase
Accuracy	Moderate	High
Robustness	Basic	Improved
Readability	Simple	Optimized

Explanation:

1. Few-shot prompting improved the output by providing examples that showed:

Upper and lowercase handling

Realistic input patterns

This helped the AI generate a more accurate and generalized solution.

Task Description-5: Few-shot Prompting (No min() function)

Prompt: Write a Python function to find the minimum of three numbers without using min().

Examples:

Input: (3, 5, 1) → Output: 1

Input: (10, 2, 7) → Output: 2

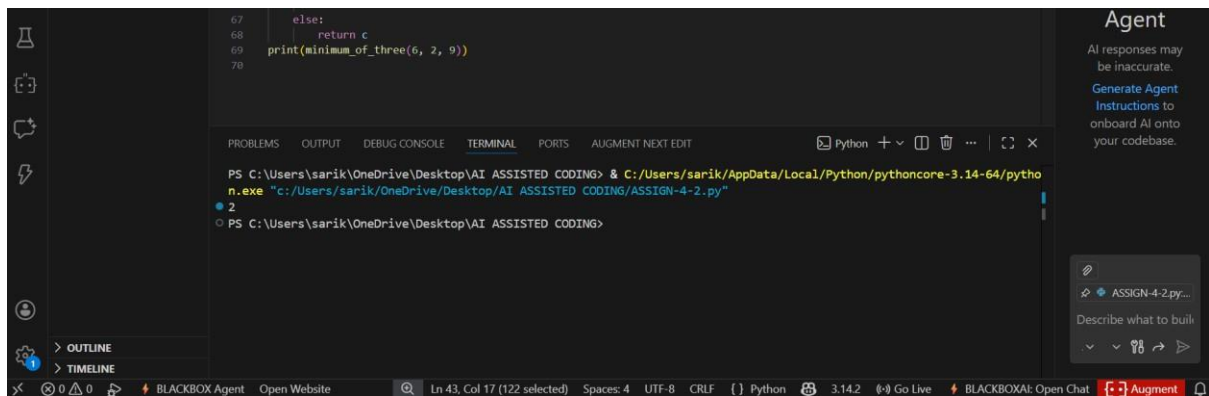
Input: (4, 4, 9) → Output: 4

```

57
58 """# Few-shot Prompting (No min() function)
59 Input: (3, 5, 1) → Output: 1
60 Input: (10, 2, 7) → Output: 2
61 Input: (4, 4, 9) → Output: 4
62 ## Write a Python function to find the minimum of three numbers without using min()."""
63 def minimum_of_three(a, b, c):
64     if a <= b and a <= c:
65         return a
66     elif b <= a and b <= c:
67         return b
68     else:
69         return c
70 print(minimum_of_three(6, 2, 9))
71

```

OUTPUT:



The screenshot shows a VS Code editor with a Python file open. The code defines a function `minimum_of_three` that takes three arguments and returns the minimum value. The terminal shows the command to run the script with arguments 6, 2, and 9, resulting in the output 2.

```
67     else:
68         return c
69     print(minimum_of_three(6, 2, 9))
70
```

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-4-2.py"
2
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

1. Few-shot examples guided logical comparisons.
2. Handles:
 - Equal values
 - All ordering cases
3. Does not use built-in `min()` as instructed.